Efficient Frontier Project Report:

For this project I have researched how to implement Linear Regression which was my method of regression, used the class example of how to optimize the portfolio as well as a youtube video for the basic skeleton of my efficient frontier. I have used a class for the monte carlo method, utilizing a risk free rate of 0.02, creating a 20000 portfolio outcome, and also utilizing returns. My class includes the weights of each portfolio, calculating the annual return, calculating the covariance, calculating annual volatility, and calculating the sharpe ratio.

I also have made my project more interactive using a while loop. I originally had idea in mind use a while loop but just a classic input function with an if, elif and else statement to limit the amount of stocks being entered to ensure only 7 stocks were inputted, however I ran into one issue that there would be an infinite loop that was created meaning that if a user entered less than 7 tickers the message “Not Enough Tickers” would infinitely print out without any stop. However if the tickers were valid and there were 7 of them the simulation would run anyways.

```
tickers = (input("Enter 7 stock tickers commas after each: ")).upper().split(",")
while len(tickers) !=7:
   if len(tickers) < 7: print("Not Enough Tickers")
   elif len(tickers) >7: print("only 7 Tickers Allowed")
   else: print("Valid Tickers", tickers)
```

This code essentially creates an infinite loop due to the input only being collected once before entering a loop. I fixed this issue by moving the input inside the while that way the input information can be recollected and eventually satisfy the condition when met. I also added a break function so that the loop can be exited.

```
      while True:
   tickers = (input("Enter 7 stock tickers commas after each: ")).upper().split(",")
   if len(tickers) < 7:
       print("Not Enough Tickers")

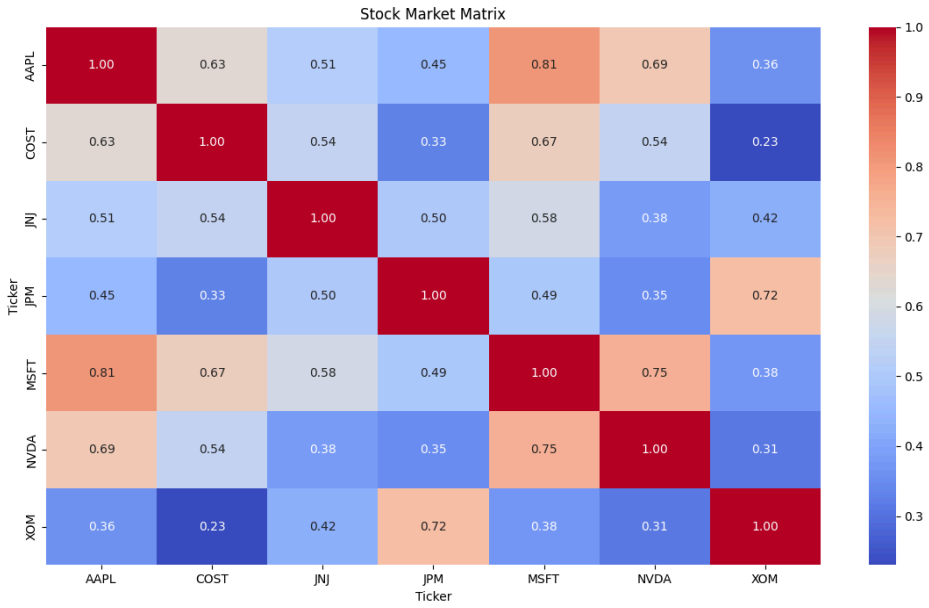
   elif len(tickers) >7:
       print("only 7 Tickers Allowed")
   else:
       print("Valid Tickers", tickers)
       break
```

I have ran as mentioned earlier a linear regression using 3 main variables: Return, Lagged Return, and volatility through sklearn. Using the the structure of a basic linear regression of:
X = df[]

Y = df[]
Model = Linear Regression
Model.fit = (X,y)

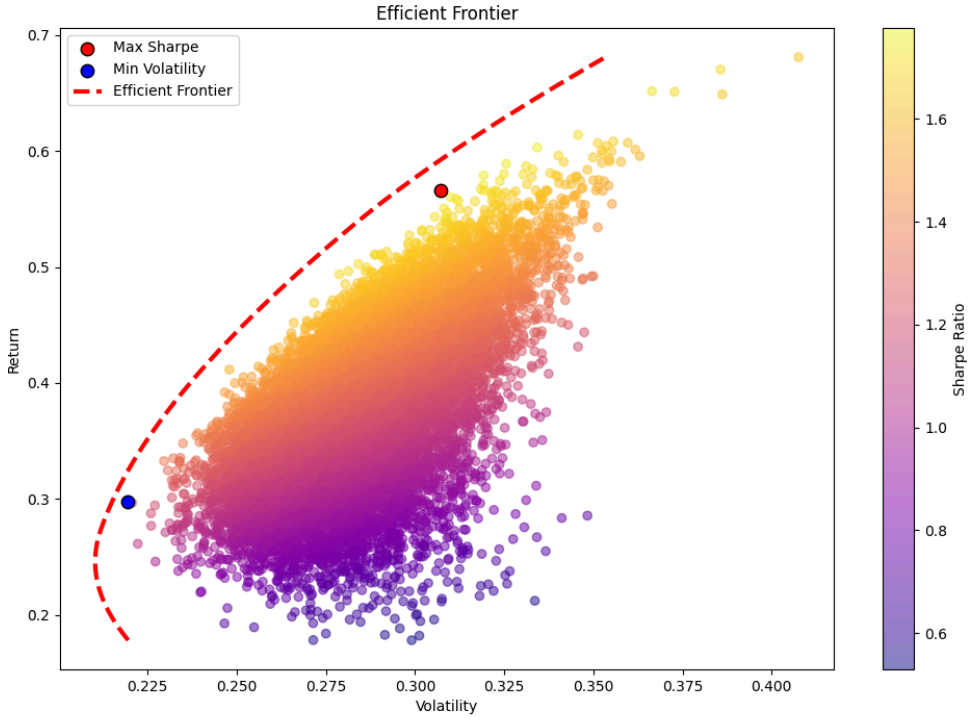
My dependent variable was returns and my independent variables were lagged returns and volatility to estimate returns.

Graphs:



This correlation heatmap shows how similarly the stocks move together and most of them do not move together well despite this portfolio of 7 stocks are tech heavy with some diversification of other sectors. We can see that MSFT and AAPL are at 0.81 in correlation and this indicates that they move similarly in the tech market, and this makes sense because they have been in the market for a long time and have stable growth year after year. The same goes for NVDA and MSFT with a correlation of 0.72 just with NVDA they have more volatile movements. JPM and XOM being financial and energy respectively respond to inflation and macroeconomic conditions more so than responding to tech movement which is why when you

compare these two stocks to the other stocks, they do not correlate as much.



This was my efficient frontier graph inspired by the class example of the output, using my 7 stock portfolio that I have chosen. These are 20,000 generated portfolios using random portfolio weights and combinations. The color combination of a brighter yellow and orange being portfolios of higher sharpe ratios and the purple/lavender colors being portfolios with lower sharpe ratio. The yellow/orange portfolios show better risk adjusted performance contrary to the purple/lavender portfolios. The red point shown in the graph is a portfolio with the maximum sharpe portfolio with the highest return relative to the risk providing a balance between return and volatility. The blue point is a portfolio with the minimum volatility, which means more stable returns and a lower expected return however. The dashed line shows the efficient frontier curve of best possible portfolios for maximum returns or minimum returns. This was created through the matplotlib library after the simulation and regressions were run, creating a scatter plot

Optimal Weights For Each Stock:

Ticker	Weight
0 AAPL	0.161286
1 MSFT	0.434414
2 JPM	0.001555

3 XOM 0.023919
4 JNJ 0.074461
5 COST 0.300319
6 NVDA 0.004044

Link to Efficient Frontier Structure:

https://youtu.be/Usxer0D-WWM?si=ZOz6MqcDCc23GxCt